LAPORAN REFLEKSI TUGAS 1
KELOMPOK 5

Disusun oleh:
I Gusti Ngurah Agung Sudarsana Antara (2505551010)
I Komang Ardi Wijaya (2505551073)

PROGRAM STUDI SARJANA TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS UDAYANA
2026

1. AI Tool yang Digunakan Dalam Mereview Tugas

Selama melakukan *review* pada kode pada fase 1, *AI tool* yang digunakan hanya Cloude AI yang dikembangkan oleh Anthropic. Hanya satu *AI tool* yang digunakan agar konteks dalam *review* dapat dikembangkan sehingga hasil terbaik dapat dilakukan.

Claude AI dipilih dalam *review* kode fase 1 sebab AI tool ini memiliki kelebihan tugas penalaran kompleks, pengodean, analisis dokumen panjang, serta percakapan alami. Hal tersebut berarti Claude AI sangat baik dalam membantu penulisan dan *debugging* kode pemrograman.

2. Saran Perubahan Kode Fase1 yang Diberikan oleh AI

Dalam proses *review* kode fase 1 yang dilakukan, terdapat beberapa saran perubahan yang diberikan oleh AI. Saran-saran yang diberikan cukup menarik karena membuka sudut pandang baru dalam memahami kode program berbasis Java.

Tabel 2.1 Saran dan Masukan Perubahan Kode Program pada Fase 1 Oleh AI

No.	Saran Perubahan	File	Folder
1	Penambahan private constructor pada kelas utility	DateUtil.java	util/
2	Penggunaan Optional sebagai nilai kembalian fungsi format()		
3	Penggunaan Optional sebagai nilai kembalian fungsi parse()		
4	Penanganan masukan format yang salah pada method parse()		
5	Perubahan nama method hariini() menjadi hariIni()		
6	Penghapusan method hariIni() karena dapat langsung menggunakan LocalDate.now()		
7	Penambahan validasi parameter method hitungSelisihHari()		
8	Penambahan private constructor pada kelas InputHelper	InputHelper.java	
9	Overload method inputInt() dengan validasi range min-max		
10	Overload method inputDouble() dengan validasi range min-max		
11	Overload method inputString() dengan parameter allowEmpty		

12	Penghapusan method closeScanner() yang berpotensi menutup System.in		
13	Penambahan method inputDate() untuk validasi format tanggal		
14	Penambahan method inputLong() untuk input tarif bertipe long		
15	Penambahan kelas baru ConsoleUtil untuk pembersihan console	ConsoleUtil.java	
16	Perubahan field status menjadi enum StatusKendaraan	Kendaraan.java	model/
17	Penambahan validasi parameter pada constructor Kendaraan		
18	Override equals() dan hashCode() pada Kendaraan		
19	Penambahan validasi pada setter setTarifPerHari()		
20	Perubahan tipe data tarifPerHari dari double menjadi long		
21	Menjadikan field id dan platNomor sebagai final		
22	Menjadikan setStatus() private, tambah method bisnis pinjam() dan kembalikan()		
23	Penambahan abstract method toJsonMap() via interface JsonSerializable		
24	Perubahan field transmisi menjadi enum Transmisi	Mobil.java	
25	Penambahan validasi parameter pada constructor Mobil		
26	Perubahan return type getJenis() menjadi enum JenisKendaraan		
27	Penambahan validasi pada setter setKapasitasPenumpang()		
28	Penambahan null-check pada setter setTransmisi()		
29	Penghapusan setter yang tidak digunakan pada Mobil		
30	Perubahan field jenisMotor menjadi enum JenisMotor	Motor.java	
31	Penambahan validasi parameter pada constructor Motor		
32	Perubahan nama field cc menjadi kapasitasCc		
33	Penambahan validasi pada setter setKapasitasCc()		
34	Penghapusan setter yang tidak digunakan pada Motor		
35	Perubahan field status menjadi enum StatusPeminjaman	Peminjaman.java	
36	Penambahan validasi parameter pada constructor Peminjaman		
37	Penambahan parameter tanggalKembaliAktual pada constructor utama		

38	Menjadikan semua setter Peminjaman private, tambah method selesaikan()		
39	Override equals() dan hashCode() pada Peminjaman		
40	Perubahan getTanggalKembaliAktual() menjadi return Optional<LocalDate>		
41	Menjadikan field immutable sebagai final pada Peminjaman		
42	Penambahan validasi urutan tanggal pada method selesaikan()		
43	Penambahan validasi parameter pada constructor User	User.java	
44	Penambahan validasi pada setter User		
45	Override equals() dan hashCode() pada User		
46	Menjadikan field id pada User sebagai final		
47	Pertimbangkan User sebagai immutable class		
48	Pembuatan enum StatusKendaraan, StatusPeminjaman, JenisKendaraan, Transmisi, JenisMotor	*.java (enum)	
49	Pembuatan interface JsonSerializable untuk memutus dependency model ke database	JsonSerializable.java	
50	Penambahan private constructor pada DataLoader	DataLoader.java	database/
51	Penambahan error handling saat parsing data numerik		
52	Penambahan penanganan jenis kendaraan tidak dikenal		
53	Penyesuaian DataLoader terhadap Optional dari DateUtil.parse()		
54	Penyesuaian DataLoader terhadap Optional dari DateUtil.format()		
55	Penambahan try-catch untuk valueOf() semua enum di DataLoader		
56	Perubahan nama method load/save menjadi muat/simpan		
57	Perbaikan nama variabel tanggal menjadi lebih deskriptif		
58	Penambahan private constructor pada FileDatabase	FileDatabase.java	
59	Penggunaan try-with-resources pada writeFile()		
60	Perubahan silent fail menjadi UncheckedIOException		
61	Penggunaan Files.readString() Java 11 pada readFile()		
61	Penggantian buildEntryNumber(double) menjadi buildEntryLong(long)		
63	Konsistensi return value null pada getValue()		

64	Pemindahan import java.util.List ke bagian atas file		
65	Pemisahan pengecekan direktori dari blok try-catch IOException		
66	Penambahan charset UTF-8 eksplisit pada FileWriter		
67	Menjadikan field dependency LaporanGenerator sebagai final	LaporanGenerator.java	report/
68	Penggunaan try-with-resources pada eksporLaporan()		
69	Perbaikan bug denda dihitung dua kali di laporanPendapatan()		
70	Perubahan tipe data totalDenda dan totalPendapatan menjadi long		
71	Penyesuaian pemanggilan DateUtil.format() terhadap Optional		
72	Penambahan validasi null pada constructor LaporanGenerator		
73	Pendelegasian logika I/O eksporLaporan() ke FileDatabase.writeFile()		
74	Perbaikan cariKendaraan() menggunakan Stream		
75	Perubahan nama method menjadi konsisten contoh adanya prefix di nama method		
76	Perubahan format printf uang dari %.0f menjadi %,d		
77	Penambahan private constructor pada DendaCalculator	DendaCalculator.java	service/
78	Perubahan return type hitung() dan hitungTotalBiaya() menjadi long		
79	Penambahan validasi parameter pada DendaCalculator		
80	Penambahan validasi status peminjaman sebelum hitung denda		
81	Penambahan Javadoc aturan bisnis minimum 1 hari pada hitungTotalBiaya()		
82	Menjadikan field users pada UserService sebagai final	UserService.java	
83	Penambahan validasi null pada constructor UserService		
84	Perubahan cariUserById() menjadi return Optional<User>		
85	Penambahan validasi duplikasi nomor telepon pada tambahUser()		
86	Penambahan pengecekan peminjaman aktif sebelum hapus user		
87	Perubahan getDaftarUser() menjadi return unmodifiableList		
88	Perbaikan generateId() menggunakan Stream		
89	Menjadikan semua field PeminjamanService sebagai final	PeminjamanService.java	

90	Penambahan validasi null pada constructor PeminjamanService		
91	Penambahan penanganan IllegalArgumentException dari valueOf()		
92	Perbaikan pemanggilan DateUtil.format() tanpa tangani Optional		
93	Perbaikan penggunaan .get() langsung pada Optional		
94	Perubahan getDaftarPeminjaman() dan getDaftarKendaraan() menjadi unmodifiableList		
95	Perubahan cariKendaraanById() dan cariPeminjamanById() menjadi return Optional		
96	Penggunaan Stream untuk getKendaraanTersedia() dan getPeminjamanAktif()		
97	Penggabungan generateIdPeminjaman() dan generateIdKendaraan() ke generic method		
98	Perbaikan unchecked cast menjadi generic type parameter di generateId()		
99	Penghapusan validasi durasi <= 0 yang redundan		
100	Penggunaan inputInt() dengan range untuk pilihan jenis kendaraan		
101	Perubahan nama kelas main menjadi Main (PascalCase)	Main.java	/
102	Penggunaan inputInt() dengan range untuk semua pilihan menu		
103	Penghapusan field static mutable, dependency diteruskan sebagai parameter		
104	Penambahan penanganan error saat load data dari file		
105	Penghapusan duplikasi tampilan peminjaman aktif di menuTransaksi()		
106	Penambahan pengecekan peminjaman aktif saat hapus user di Main		
107	Penghapusan atau anotasi default case yang tidak tercapai		
108	Penambahan konfirmasi sebelum keluar program		
109	Pemisahan tampilan header ke method tersendiri tampilkanHeader()		
110	Penggunaan enhanced switch Java 14+ untuk keterbacaan		

Tabel tersebut menunjukkan seluruh saran atau masukan yang diberikan oleh AI selama sesi *review*. Tidak semua saran atau masukan yang diberikan diterapkan untuk memperbaiki kode pada fase 1. Terdapat beberapa saran yang tidak diterapkan di fase 2.

3. Saran yang Diterima

Secara keseluruhan, kami menerima sebagian besar saran yang diberikan karena saran-saran tersebut memperkuat kualitas kode secara nyata dan dapat dirasakan manfaatnya langsung saat mengimplementasikan perubahan.

Saran pertama yang diterima dengan penuh keyakinan adalah penggunaan *enum* menggantikan *String* untuk konstanta terbatas seperti StatusKendaraan, StatusPeminjaman, Transmisi, dan JenisMotor. Setelah memahami bahwa *String* sangat rentan terhadap kesalahan penulisan yang tidak terdeteksi saat kompilasi, kami menyadari bahwa *enum* memberikan keamanan tipe yang jauh lebih baik. *Compiler* langsung mendeteksi kesalahan yang sebelumnya baru terlihat saat *runtime*, dan IDE memberikan *auto-complete* yang memudahkan penulisan kode. Ini adalah perubahan yang terasa paling signifikan karena menyentuh hampir seluruh lapisan aplikasi.

Saran berikutnya yang diterima adalah penambahan validasi parameter di *constructor* seluruh kelas model. Sebelumnya kami tidak menyadari bahwa objek bisa terbentuk dengan data yang tidak valid seperti tarif negatif atau ID kosong. Setelah memahami prinsip *fail fast* dari *Effective Java Item* 49, kami setuju bahwa lebih baik program gagal di titik pembentukan objek dengan pesan yang jelas daripada menghasilkan data korup yang baru diketahui jauh kemudian di lapisan lain.

Saran tentang *private constructor* untuk *utility class* juga diterima karena alasannya sangat masuk akal. Kelas seperti DateUtil, InputHelper, dan DendaCalculator tidak memiliki *state* apapun dan tidak bermakna untuk diinstansiasi. Menambahkan private constructor dengan AssertionError adalah cara yang eksplisit untuk mengkomunikasikan keputusan desain ini kepada pembaca kode di masa depan.

Perubahan *setStatus()* menjadi *private* dan penambahan *method* bisnis pinjam(), kembalikan(), dan selesaikan() adalah saran yang awalnya dipertanyakan, namun akhirnya kami menerimanya setelah memahami konsep enkapsulasi yang sesungguhnya. Kami menyadari bahwa enkapsulasi bukan sekadar menjadikan

field private lalu menyediakan *getter* dan *setter*, melainkan menyembunyikan detail implementasi dan mengekspos hanya operasi yang bermakna secara bisnis. *Method* pinjam() jauh lebih ekspresif dan aman dibandingkan setStatus(StatusKendaraan.DIPINJAM) karena juga memvalidasi *state* sebelumnya.

Saran *constructor chaining* untuk pemisahan antara pembuatan objek baru dan rekonstruksi dari *database* juga diterima karena ini menyelesaikan masalah nyata. Tanpa ini, *setter* harus tetap *public* untuk keperluan DataLoader, yang justru membuka celah enkapsulasi dari luar. Dengan *constructor chaining*, semua validasi terpusat di satu tempat dan tidak ada duplikasi logika.

Penggunaan Optional untuk nilai yang mungkin tidak ada seperti getTanggalKembaliAktual() dan *return value* DateUtil.parse() juga diterima setelah memahami bahwa *null* yang mengalir diam-diam antar lapisan adalah sumber *bug* yang sulit dilacak. Optional memaksa pemanggil untuk sadar dan menangani kemungkinan tidak adanya nilai secara eksplisit, sehingga NullPointerException yang tidak terduga bisa dicegah sejak awal.

Saran Collections.unmodifiableList() untuk *getter* yang mengembalikan koleksi internal diterima karena kami sadar bahwa mengembalikan referensi langsung ke list internal memungkinkan siapapun memodifikasi data tanpa melalui kontrol yang semestinya. Ini adalah celah enkapsulasi yang halus namun berbahaya.

Perubahan tipe data *double* ke *long* untuk merepresentasikan uang juga diterima setelah memahami ketidakpresisian *floating-point* untuk nilai desimal. Menggunakan *long* dalam satuan rupiah penuh jauh lebih aman untuk konteks aplikasi ini dan menghilangkan risiko nilai seperti 299999.9999 yang seharusnya 300000.

Saran *interface* JsonSerializable dengan *method* toJsonMap() juga diterima setelah kami memahami bahwa ketergantungan model ke FileDatabase adalah pelanggaran arsitektur yang serius. Model seharusnya tidak tahu cara datanya disimpan karena itu adalah tanggung jawab lapisan *database*. Dengan *interface* ini, *dependency* mengalir satu arah dan model menjadi benar-benar independen dari format penyimpanan.

Untuk UncheckedIOException menggantikan *silent fail* di FileDatabase, diterima karena menelan *exception* dan melanjutkan program seolah tidak ada masalah adalah hal yang sangat berbahaya. Data bisa hilang tanpa peringatan apapun, sehingga lebih baik program berhenti dengan pesan yang jelas daripada diam-diam kehilangan data.

Masih banyak saran atau masukan yang diterima untuk perubahan kode program fase 2. Contohnya adalah penamaan *method* yang konsisten dari segi bahasa dan awalan *prefix*. Selain itu, masih banyak lagi saran atau masukan yang diterima tetapi tidak dapat dijelaskan semuanya dalam bab ini.

4. Saran yang Ditolak

Saran penambahan *method* inputDate() di InputHelper tidak diterima karena dalam sistem ini input tanggal tidak dilakukan secara langsung oleh *user*. Durasi pinjam diinput sebagai angka hari, bukan tanggal, sehingga *method* ini tidak memiliki penggunaan yang nyata dalam aplikasi ini.

Saran menjadikan User sebagai *immutable class* sepenuhnya tidak diterima karena pertimbangkan kemungkinan fitur edit data user di masa depan. Menghapus kesempatan penggunaan *setter* sepenuhnya akan mempersulit pengembangan selanjutnya meskipun untuk saat ini memang belum ada fitur tersebut.

Saran pendelegasian logika I/O eksporLaporan() ke FileDatabase.writeFile() tidak diterima karena ekspor laporan memiliki kebutuhan penanganan *error* yang berbeda. Saat ekspor gagal, program tidak perlu berhenti seperti UncheckedIOException yang dilempar FileDatabase. Ekspor laporan adalah fitur opsional yang kegagalannya cukup ditampilkan sebagai pesan peringatan.

Saran konfirmasi sebelum keluar tidak diterima karena untuk kode sudah berjalan cukup baik tanpa perlu hal itu. Aplikasi ini sudah cukup jelas alur keluarnya sehingga tidak diperlukan konfirmasi untuk keluar.

Saran *enhanced switch* Java 14+ tidak diterima karena proyek ini menargetkan Java 11 sebagai minimum requirement, sehingga menggunakan fitur

Java 14+ akan mengurangi kompatibilitas tanpa manfaat yang berarti untuk skala proyek ini.

5. Saran yang Salah atau Kurang Tepat dari AI

Selama sesi *review* dalam satu sesi obrolan, tidak ada saran atau masukan yang kurang tepat dari AI. Seluruh saran dan masukan diberikan berdasarkan pengamatan yang sangat baik dari AI.

Meski begitu, tidak semua saran atau masukan dari AI diterapkan. Seperti yang sudah di jelaskan pada bab sebelumnya, ada beberapa alasan mengapa saran dari AI tidak diterapkan ke dalam fase 2.

6. Konsep OOP yang Dipahami Setelah Melakukan Review

Setelah proses *review* dengan AI, kami menjadi lebih memahami bahwa konsep OOP tidak hanya sebatas menggunakan *class*, *inheritance*, atau *abstract* class, tetapi juga tentang bagaimana merancang objek yang baik, aman, dan mudah dikelola. Sebelumnya kami merasa sudah cukup memahami OOP karena telah menggunakan berbagai fitur dasarnya. Namun, setelah kode ditinjau lebih dalam, kami menyadari masih banyak konsep yang belum diterapkan secara maksimal.

Konsep yang paling dipahami adalah enkapsulasi. Awalnya kami menganggap enkapsulasi sudah cukup jika field dibuat private lalu disediakan *getter* dan *setter*. Setelah *review*, kami memahami bahwa enkapsulasi berarti melindungi data dan mengatur perubahan *state* objek agar hanya bisa dilakukan melalui proses yang valid. Contohnya, perubahan status kendaraan lebih baik dilakukan melalui method seperti pinjam(), kembalikan(), dan selesaikan(), bukan melalui *setter* biasa. Dengan cara ini, objek menjadi lebih aman dan logika bisnis lebih jelas.

Kami juga lebih memahami fungsi *constructor* dalam OOP. Sebelumnya *constructor* hanya kami gunakan untuk mengisi nilai awal atribut. Setelah *review*, kami memahami bahwa *constructor* seharusnya menjadi tempat validasi data agar objek tidak bisa dibuat dalam kondisi tidak valid. Misalnya, ID kosong atau tarif

bernilai negatif harus langsung ditolak saat objek dibuat. Pendekatan ini membuat kesalahan dapat diketahui lebih awal dan mencegah *bug* di bagian lain program.

Selain itu, kami menjadi lebih memahami *abstract class*. Sebelumnya kami menggunakannya hanya sebagai bentuk pewarisan. Setelah *review*, kami memahami bahwa *abstract class* berfungsi sebagai kontrak bagi *subclass*. Jika ada method *abstrak*, maka setiap turunan class wajib mengimplementasikan perilaku tersebut. Dengan demikian, struktur program menjadi lebih konsisten.

Konsep lain yang semakin jelas adalah prinsip SOLID, terutama *Single Responsibility Principle* dan *Open Closed Principle*. Sebelumnya konsep ini terasa seperti teori. Namun, setelah melihat langsung pada kode sendiri, kami memahami bahwa satu *class* sebaiknya memiliki satu tanggung jawab utama, dan sistem sebaiknya mudah dikembangkan tanpa harus banyak mengubah kode lama.

Secara keseluruhan, *review* dengan AI membuat kami lebih memahami bahwa inti OOP adalah menjaga integritas objek, membagi tanggung jawab dengan jelas, serta membuat kode lebih terstruktur dan mudah dikembangkan. Jadi, OOP bukan hanya tentang penggunaan *class* dan *inheritance*, tetapi juga tentang desain perangkat lunak yang baik.

7. Pembagian Tugas

Dalam pengerjaan tugas pertama ini, kami sepakat untuk membagi kerja sesuai fase. Fase pertama yaitu pengerjaan kode secara mandiri tanpa bantuan AI dikerjakan oleh I Komang Ardi Wijaya (dengan NIM 2505551073). Fase kedua yaitu *review* kode program pada fase pertama untuk mencari saran atau masukan perubahan kode program dikerjakan oleh I Gusti Ngurah Agung Sudarsana Antara (dengan NIM 2505551010).

Pembagian dilakukan seperti itu karena pertimbangan kami tentang waktu untuk mengerjakan tugas ini bersama. Karena kami tidak menemukan kecocokan jadwal di antara satu dengan yang lain, kami memutuskan untuk membagi tugas sesuai dengan fase. Hal ini dimaksudkan agar kode program yang dihasilkan dapat fokus pada pemikiran satu orang sehingga tidak ada kode program yang tidak konsisten.

Tabel 7.1 Pembagian Tugas Anggota Kelompok

No.	Nama	NIM	Tugas pada fase Pertama	Tugas pada Fase Kedua
1	I Gusti Ngurah Agung Sudarsana Antara	2505551010	Tidak ada	Keseluruhan pengerjaan
2	I Komang Ardi Wijaya	2505551073	Keseluruhan oengerjaan	Tidak ada

Tabel tersebut menunjukkan bagaimana pembagian tugas kelompok dilakukan. Meskipun tidak terlibat langsung di kedua fase, masing-masing anggota dapat memahami bagaimana konsep OOP dalam Java bekerja.